Highlights

RepoAI: Automated Code Refactoring through Multi-Agent LLM Orchestration and Retrieval-Augmented Generation

Moe Pyae Pyae Kyaw, Soe Moe Ko, Pyae Phyo Paing, Min Khant Than Swe, Tew Hongthong, Nacha Chondamrongkul

- Multi-agent architecture for coordinated repository-level code refactoring using LLMs.
- Retrieval-Augmented Generation pipeline for context-aware code understanding and modification.
- Comprehensive validation framework combining syntax, semantic, and intent verification.
- Automated GitHub integration for seamless deployment through pull request workflows.
- Empirical evaluation demonstrating effectiveness and limitations of LLM-based refactoring.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

RepoAI: Automated Code Refactoring through Multi-Agent LLM Orchestration and Retrieval-Augmented Generation

Moe Pyae Pyae Kyaw[a], Soe Moe Ko[a], Pyae Phyo Paing[a], Min Khant Than Swe[a], Tew Hongthong[a], Nacha Chondamrongkul[a,*]

[a]*School of Applied Digital Technology, Mae Fah Luang University, Chiang Rai, Thailand*

Abstract

While Large Language Models have demonstrated strong code generation capabilities, existing approaches operate at single-file level without repository-wide context or systematic validation. This paper presents RepoAI, a multi-agent system that automates repository-level code refactoring through coordinated LLM orchestration. Our approach employs specialised agents for intent classification, code generation, and validation, working collaboratively to handle complex multi-file modifications. The system combines Retrieval-Augmented Generation for context-aware code understanding with a comprehensive validation pipeline ensuring correctness before automated GitHub deployment. Experimental evaluation demonstrates that multi-agent coordination significantly improves refactoring accuracy for localised modifications, while RAG-based retrieval improves intent verification by 27% over context-free generation.

Keywords: Code Refactoring, Large Language Models, Multi-Agent Systems, Retrieval-Augmented Generation, Software Maintenance, GitHub Automation

1. Introduction

The rapid growth of large-scale software systems has created an urgent need for intelligent tools that support code comprehension, refactoring, and

*Corresponding author
Email address: nacha.cho@mfu.ac.th (Nacha Chondamrongkul)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

maintenance. Modern developers frequently interact with unfamiliar repositories, navigate complex directory structures, and attempt to modify codebases without breaking existing functionality. These tasks are not only time-consuming but also highly error-prone, particularly when developers must work across multiple files, resolve inter-file dependencies, or ensure consistent architectural patterns. As software development teams increasingly rely on distributed, collaborative workflows, the demand for automated assistance in repository understanding and safe code transformation continues to rise.

Recent advancements in Large Language Models (LLMs) have demonstrated strong capabilities in reasoning over source code, generating new functions, and providing natural-language explanations of technical concepts [7]. However, most LLM-based code assistants still operate at the level of isolated files or single-function generation. They lack the ability to ingest entire codebases, understand project-wide structures, plan multi-file modifications, or validate whether newly generated code preserves logical and functional intent. Additionally, integrating LLM-driven code generation directly into version control platforms—such as GitHub—remains a significant challenge, requiring reliable mechanisms for branch creation, file updates, conflict avoidance, and automated pull request generation.

1.1. Challenges in LLM-Based Code Refactoring

Several challenges must be addressed to enable effective LLM-based code refactoring at the repository level. The most fundamental challenge concerns context limitation: LLMs have finite context windows that cannot accommodate entire repositories, and without mechanisms to selectively retrieve relevant code segments, the model lacks sufficient context to reason about cross-file dependencies and architectural patterns [2]. This limitation is compounded by the challenge of multi-file coordination, as repository-level refactoring often requires coordinated modifications across multiple files while maintaining consistency in imports, function signatures, and module interfaces. Single-agent LLM systems struggle to decompose such complex workflows into manageable subtasks [7].

Beyond coordination, validation and correctness present significant obstacles. LLMs may generate syntactically valid but semantically incorrect code, or produce outputs that fail to satisfy the user's original intent, making comprehensive validation mechanisms essential to detect such errors before deployment [5, 6]. Closely related is the challenge of hallucination and reliability: LLMs may produce plausible but incorrect code, particularly when

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

operating without grounded context from the actual repository [3, 4], necessitating retrieval mechanisms that anchor generation in actual code artifacts. Finally, integration with development workflows requires seamless connection with version control systems, including branch management, commit generation, and pull request creation, while maintaining audit trails for review.

1.2. Contributions

To address the limitations of existing code assistants, we present **RepoAI**, a web-based intelligent system that combines Retrieval-Augmented Generation (RAG), multi-agent orchestration, and comprehensive validation for automated code refactoring. The contributions of this work span architectural design, methodological innovation, and empirical investigation.

First, we introduce a multi-agent architecture built on Microsoft AutoGen that coordinates specialised agents for distinct phases of the refactoring workflow. The architecture separates concerns across agents responsible for intent classification, repository analysis, multi-file code generation, validation orchestration, and GitHub integration, enabling independent optimisation of each component while maintaining coherent end-to-end operation.

Second, we develop a RAG pipeline specifically designed for code repositories that constructs vectorised knowledge spaces from GitHub repositories using state-of-the-art embedding models. This pipeline enables context-aware retrieval of relevant code segments during generation, grounding LLM outputs in actual repository content and substantially reducing hallucination compared to context-free approaches.

Third, we present a comprehensive validation framework that addresses the reliability challenges inherent in LLM-based code generation. The framework combines complementary validation strategies including syntax checking through language-specific parsers, semantic preservation analysis comparing structural elements between original and modified code, embedding-based similarity scoring for intent alignment assessment, and LLM-based verification that evaluates whether generated code fulfills user requirements.

Fourth, we implement automated GitHub integration that bridges the gap between code generation and deployment within standard development workflows. This integration handles the complete lifecycle from authenticated repository access through branch creation, file modification, and pull request generation with appropriate metadata for human review.

Finally, we contribute an empirical evaluation that demonstrates both the effectiveness and limitations of the proposed approach across multiple

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

refactoring tasks and programming languages, providing guidance for practitioners considering adoption and researchers pursuing further advances in this area.

The remainder of this paper is organised as follows. Section 2 discusses background concepts and related work. Section 3 presents the system architecture and design including the multi-agent framework and validation pipeline. Section 4 presents the experimental evaluation and results. Section 5 discusses findings and limitations. Finally, Section 6 concludes the paper and outlines future research directions.

2. Background and Related Work

This section presents the background concepts underlying our approach and discusses related work in code refactoring automation, LLM-based code generation, and retrieval-augmented generation.

2.1. Large Language Models for Code

Large Language Models have become increasingly capable in code-related tasks, including code generation, completion, summarisation, and defect identification [7]. Models such as GPT-4, Claude, and Gemini demonstrate strong performance on programming benchmarks by learning patterns from large codebases. The development of specialised code models has accelerated significantly, with CodeLlama [12] achieving state-of-the-art results by fine-tuning Llama 2 on code-specific datasets across multiple programming languages. Similarly, the BigCode project's StarCoder and StarCoder2 models [13] trained on permissively licensed code from The Stack demonstrate competitive performance while maintaining open-source accessibility. DeepSeek-Coder [14] further advances the field with models trained on 2 trillion tokens across 87% code and 13% natural language, achieving performance that surpasses earlier closed-source models on benchmarks including HumanEval and MBPP.

These models are typically evaluated using standardised benchmarks. HumanEval [15], developed by OpenAI, measures functional correctness using a pass@k metric that evaluates whether generated code passes unit tests. MBPP (Mostly Basic Python Problems) provides complementary evaluation with a larger set of programming tasks. Recent extensions including HumanEval Pro and MBPP Pro [16] introduce self-invoking code generation tasks that better assess progressive reasoning capabilities.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

However, these models face inherent limitations when applied to repository-level tasks. The context window constraint prevents processing entire codebases, while the lack of grounding in actual repository content can lead to hallucinated outputs that reference non-existent modules or incorrect APIs [3, 4]. Several approaches have been proposed to extend LLM capabilities for code. Chain-of-Thought prompting [8] improves reasoning by decomposing problems into intermediate steps. Tree of Thoughts [9] enables exploration of multiple solution branches. However, these techniques primarily enhance single-query reasoning rather than addressing repository-level coordination.

2.2. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [2] addresses the context limitation of LLMs by retrieving relevant information from external knowledge sources before generation. In code domains, RAG enables retrieval of relevant code segments, documentation, or API specifications that provide grounding context for generation. This approach reduces hallucination by anchoring LLM outputs in actual artifacts from the target repository.

The effectiveness of RAG depends critically on the quality of code embeddings used for retrieval. CodeBERT [17] introduced bimodal pre-training on programming and natural languages, learning deep code semantics through masked language modelling and replaced token detection. GraphCodeBERT [18] extends this approach by incorporating data flow graphs to capture structural dependencies, improving performance on code search, clone detection, and code refinement tasks. These embedding models enable semantic matching between natural language queries and code segments, allowing retrieval to surface contextually relevant code even when queries use different terminology than the indexed content.

The RAG pipeline typically consists of three stages: (1) indexing, where documents are embedded into vector representations; (2) retrieval, where queries are matched against the vector index to find relevant segments; and (3) generation, where retrieved context augments the LLM prompt. For code repositories, effective chunking strategies must preserve semantic coherence of functions and classes while fitting within embedding model constraints. Recent work on retrieval-augmented program repair, such as RelRepair [19], demonstrates that retrieving relevant function signatures and code snippets from the codebase can significantly improve LLM-generated patches.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

2.3. Multi-Agent LLM Systems

Multi-agent architectures address the limitation of single-agent systems in handling complex, multi-step workflows. Microsoft AutoGen provides a framework for orchestrating specialised agents that collaborate through message passing [7]. Each agent can be configured with specific capabilities, tools, and prompts optimised for particular subtasks. LangChain and its successor LangGraph [20] have emerged as popular frameworks for building production-ready agents, with adoption by companies including LinkedIn, Uber, and GitLab for tasks ranging from SQL generation to large-scale code migrations.

Several multi-agent frameworks specifically target software engineering tasks. ChatDev [21] simulates a software company with agents taking roles such as CEO, CTO, programmer, and tester, while MetaGPT encodes standard operating procedures into multi-agent task prompts. More recently, MAGIS [22] demonstrates a multi-agent framework for GitHub issue resolution that integrates code generation into a multi-step process with Developer and QA Engineer agents. These frameworks share the insight that dividing labour among specialised agents can overcome individual LLM limitations through structured collaboration.

In the context of code refactoring, multi-agent systems enable decomposition of complex tasks into specialised roles: intent classification, repository analysis, code generation, validation, and deployment. This separation of concerns improves maintainability and allows independent optimisation of each component. However, challenges remain including high token costs, instability in iteration, and difficulty generalising to complex software projects.

2.4. Automated Code Refactoring

Automated refactoring has been studied extensively in software engineering research. Traditional approaches rely on rule-based transformations or search-based algorithms to identify and apply refactoring patterns [1]. Tools such as SonarQube [11] detect code smells and suggest improvements, while IDE-integrated refactoring tools apply mechanical transformations such as rename, extract method, or move class.

The closely related field of automated program repair (APR) has seen significant advances through deep learning. Learning-based APR techniques treat bug fixing as a neural machine translation task, where buggy code is translated to fixed code [23]. Recent approaches leverage LLMs directly for patch generation; ChatRepair [24] uses ChatGPT's conversational interface

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

to iteratively incorporate feedback from failing tests, achieving state-of-the-art results on the Defects4j benchmark with 162 correct fixes out of 337 bugs. These techniques demonstrate that LLMs can effectively learn bug-fixing patterns when provided with appropriate context and feedback.

More recently, LLM-based approaches have been proposed for intelligent code transformation. GitHub Copilot [10] provides inline suggestions during development, while ChatGPT and similar assistants can generate refactored code in response to natural language requests. However, these tools typically lack repository-level context awareness and systematic validation of generated outputs.

The review of existing approaches reveals a significant gap in current LLM-based code assistance tools. While individual techniques for code generation, retrieval augmentation, and multi-agent coordination have been explored, no existing system combines these capabilities into an integrated solution for repository-level refactoring. Current tools either operate at the single-file level without cross-file awareness, lack systematic validation of generated outputs, or fail to integrate with standard development workflows through automated pull request generation. This gap motivates the development of RepoAI, which addresses these limitations through a unified architecture combining RAG-based retrieval, multi-agent orchestration, comprehensive validation, and automated GitHub integration.

3. RepoAI: Architecture and Implementation

This section presents the architecture and implementation of RepoAI[1], organised around the key technical components: the retrieval-augmented generation pipeline for context acquisition, the multi-agent architecture for task coordination, the validation framework for quality assurance, and the GitHub integration for automated deployment.

3.1. Architecture Overview

RepoAI is designed as an AI-powered GitHub repository refactoring system constructed using Streamlit, LangChain, and Google Vertex AI. The system integrates autonomous agents capable of interpreting natural language queries, analysing multi-file software repositories, and generating validated

[1]The source code of RepoAI is available at `https://github.com/Meo-6531503185/Repo-AI`.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

refactorings deployed through automated pull requests. Figure 1 illustrates the multi-agent architecture and data flow.

The choice of underlying language model significantly impacts system capabilities. We selected **Gemini 2.5 Pro** as the primary LLM for several reasons. First, its extended context window of up to one million tokens enables processing of entire repository contents within a single context, which is essential for understanding cross-file dependencies and maintaining consistency across multi-file refactoring operations. Second, Gemini 2.5 Pro demonstrates strong performance on code-related benchmarks, including HumanEval and MBPP, indicating robust code generation and comprehension capabilities. Third, the model exhibits advanced reasoning abilities necessary for decomposing complex refactoring requests into coherent sequences of file operations. Fourth, integration through Google Vertex AI provides enterprise-grade reliability, scalability, and security compliance required for processing potentially sensitive codebases.

For semantic retrieval, we employ **Vertex AI's text-embedding-004** model to generate vector representations of code chunks. This embedding model was selected for its strong performance on semantic similarity tasks and its optimisation for technical content including source code. The 768-dimensional embeddings capture nuanced semantic relationships between code segments, enabling accurate retrieval of contextually relevant code even when queries use different terminology than the indexed content. The model's integration within the Vertex AI ecosystem simplifies deployment and ensures consistent performance characteristics across the retrieval and generation components.

The architecture centres on a **ReasoningAgent** that serves as the central orchestrator, receiving user requests through the Streamlit UI and routing them to appropriate specialised agents. When a user submits a query, the ReasoningAgent analyses the intent and classifies it as either a repository overview request or a refactoring operation. For overview requests, the **OverviewAgent** is invoked, which employs four internal components: QueryRouter for intent classification, VectorSearch for semantic retrieval against the indexed codebase, RepoAnalyzer for structural analysis, and ResponseGen for natural language explanation generation.

For refactoring requests, the **MultiFileRefactorAgent** handles code generation, coordinating file analysis and producing modified code using Gemini 2.5 Pro. The generated code then passes through the **ValidationPipeline**, which executes four complementary validators: Code Quality for

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

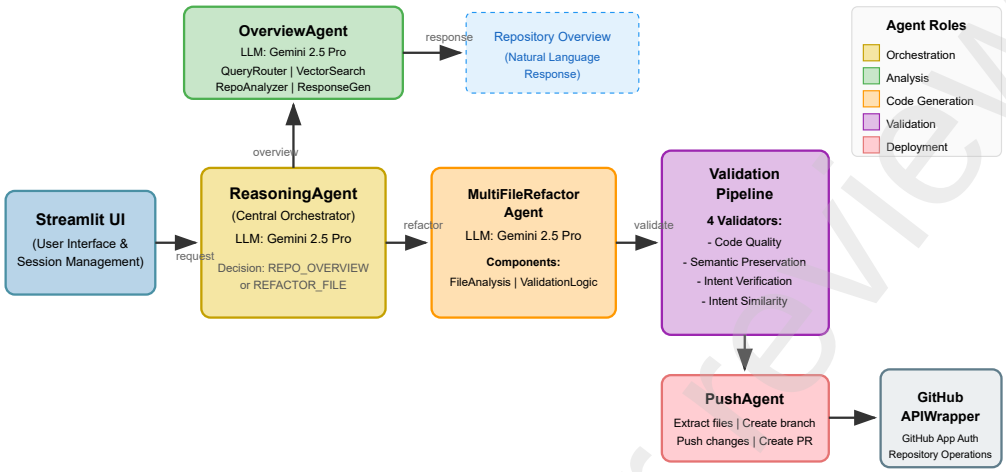


Figure 1: Multi-agent architecture of RepoAI showing the coordination between specialised agents

syntax checking, Semantic Preservation for interface compatibility, Intent Verification for requirement alignment, and Intent Similarity for embedding-based consistency checking.

Finally, validated changes flow to the **PushAgent**, which extracts files from LLM outputs, creates feature branches, commits modifications, and generates pull requests through the **GitHubAPIWrapper**. This wrapper handles GitHub App authentication and provides secure, scoped access to repository operations. The multi-agent design enables each component to be optimised independently while maintaining coherent end-to-end operation through structured message passing.

3.2. Retrieval-Augmented Generation Pipeline

The RAG pipeline enables context-aware code understanding by grounding LLM generation in actual repository content rather than relying solely on the model's parametric knowledge acquired during pretraining. This grounding is essential for repository-level tasks where the LLM must reference specific implementations, follow established conventions, and integrate with existing interfaces that it has never encountered during training.

The pipeline begins when a user provides a GitHub repository URL. The system extracts the owner and repository name, then authenticates using

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

GitHub App credentials to obtain a temporary installation token. Using this token, the system recursively traverses the repository structure and fetches the content of all source files. Each file undergoes intelligent chunking that divides content into segments of at most 20,000 tokens while maintaining 200 tokens of overlap between adjacent chunks to preserve context continuity across boundaries. These chunks are then processed through Vertex AI's `text-embedding-004` model to generate high-dimensional vector representations capturing semantic meaning. The resulting embeddings are organised into a FAISS vector store optimised for efficient similarity search, enabling rapid retrieval of relevant code segments during subsequent operations. Algorithm 1 formalises this retrieval process.

Algorithm 1 RAG-based code retrieval

Require: Repository URL R, User query Q, Top-k parameter k
Ensure: Relevant code segments S

```
files ← FetchRepository(R)
chunks ← ∅
for each file ∈ files do
    c ← ChunkFile(file, maxTokens = 20000, overlap = 200)
    chunks ← chunks ∪ c
embeddings ← GenerateEmbeddings(chunks)
vectorStore ← BuildFAISS(embeddings)
queryEmbed ← GenerateEmbedding(Q)
S ← SimilaritySearch(vectorStore, queryEmbed, k)
return S
```

The chunking strategy balances competing concerns of preserving semantic coherence and fitting within embedding model constraints. Where possible, chunk boundaries align with natural code structure such as function definitions and class declarations, ensuring that retrieved segments contain complete logical units rather than arbitrary fragments. The 200-token overlap between adjacent chunks provides continuity that allows retrieval to surface context even when relevant content spans chunk boundaries. This overlap is particularly important for understanding function implementations that reference variables or helper functions defined earlier in the same file.

The choice of FAISS as the vector store backend reflects the need for efficient similarity search over potentially large embedding collections. FAISS provides sub-linear search complexity through approximate nearest neighbour algorithms, enabling responsive retrieval even for repositories containing thousands of files. The top-k parameter controls the trade-off between

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

context comprehensiveness and prompt length, with higher values providing more context at the cost of increased token consumption and potential noise introduction.

3.3. Multi-Agent Architecture

The multi-agent architecture implements coordinated reasoning using Microsoft AutoGen's framework, where specialised agents communicate through asynchronous message passing. This design reflects the insight that different phases of repository refactoring require qualitatively different reasoning capabilities: intent classification demands linguistic analysis, code generation requires programming expertise, and validation needs systematic checking procedures. Figure 2 illustrates the coordination protocol and information exchange between agents.

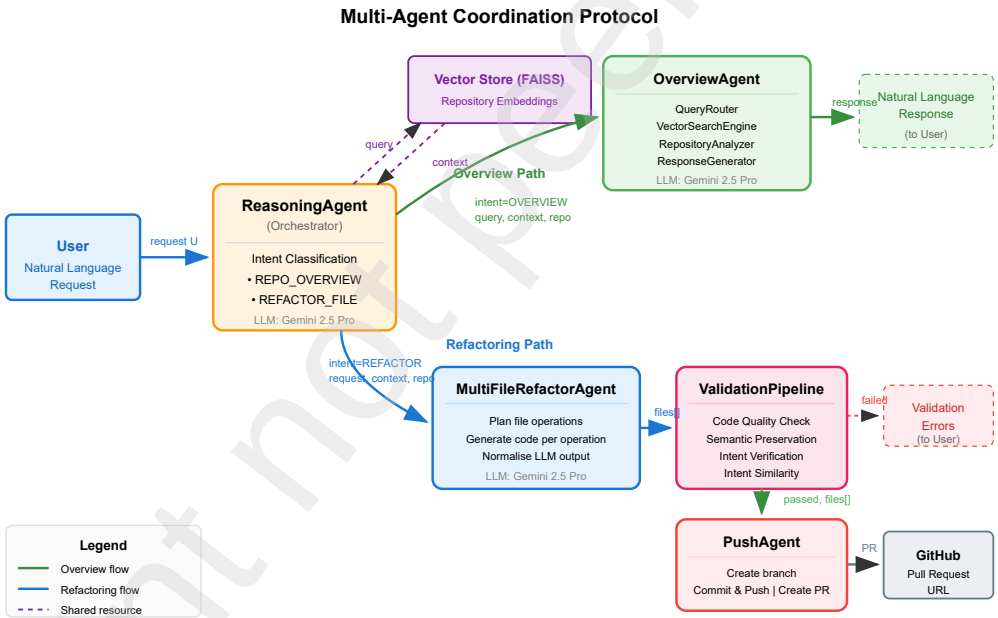


Figure 2: Multi-agent coordination protocol showing the two processing paths: the overview path (green) for repository exploration queries, and the refactoring path (blue) for code modification requests. Arrows indicate the direction and type of information exchanged between agents.

The **ReasoningAgent** serves as the central orchestrator, interpreting user intent and delegating tasks to appropriate sub-agents. It classifies

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

queries into two categories: `REPO_OVERVIEW` for repository exploration and information retrieval, or `REFACTOR_FILE` for code modification requests. Classification is performed using prompt-based LLM reasoning with Gemini 2.5 Pro, analysing linguistic structure and semantic content to identify action verbs, interrogative patterns, and domain-specific terminology.

The **OverviewAgent** handles repository-level analysis and natural language question answering through four coordinated internal components. The QueryRouter classifies incoming queries into distinct types such as file-specific inquiries, architectural questions, or general overview requests. The VectorSearchEngine performs semantic retrieval against indexed repository content using Vertex AI embeddings, with automatic fallback to keyword-based search when semantic matching yields insufficient results. The RepositoryAnalyzer reconstructs project hierarchy, identifies key entry points, and extracts architectural patterns. Finally, the ResponseGenerator synthesises information from retrieved segments and structural analysis to produce natural language explanations tailored to the query type.

The **MultiFileRefactorAgent** handles cross-file reasoning and automated refactoring using structured representations to describe file operations (create, modify, delete, rename). The agent manages import paths, module dependencies, and project structure detection across multiple programming languages including Python, Java, JavaScript, TypeScript, Go, C/C++, Rust, and Ruby. For each operation, it constructs specialised prompts incorporating retrieved context, project structure information, and language-specific guidance.

The **PushAgent** manages GitHub interactions, extracting files from LLM outputs, determining appropriate file paths, and pushing updates through branch creation, commit generation, and pull request submission.

Algorithm 2 formalises the coordination protocol that orchestrates these agents. As shown in the algorithm, the ReasoningAgent first classifies user intent (line 1), then delegates to either the OverviewAgent for information requests (lines 2–4) or the MultiFileRefactorAgent for refactoring operations (lines 5–16). The refactoring path includes planning, code generation with normalisation, validation, and conditional deployment.

The planning phase of refactoring deserves particular attention, as it determines the structure of subsequent generation operations. When the MultiFileRefactorAgent receives a modification request, it first analyses the request to identify all required file operations, categorising each as a creation of new files, modification of existing files, deletion of obsolete files, or renaming of

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

Algorithm 2 Multi-agent task coordination

Require: User request U, Repository data R, Vector store V
Ensure: Response or Pull request URL

```
intent ← ReasoningAgent.Classify(U)
if intent = REPO_OVERVIEW then
    context ← VectorSearch(V, U)
    response ← OverviewAgent.Generate(U, context, R)
    return response
else if intent = REFACTOR_FILE then
    context ← VectorSearch(V, U)
    operations ← RefactorAgent.Plan(U, context, R)
    files ← ∅
    for each op ∈ operations do
        code ← RefactorAgent.Generate(op, context)
        code ← Normalise(code)
        files ← files ∪ {(op.path, code)}
    validationResult ← ValidatePipeline(files)
    if validationResult.passed then
        prUrl ← PushAgent.CreatePR(files)
        return prUrl
    else
        return validationResult.errors
```

files to new paths. This categorisation enables execution in a specific order designed to maintain consistency throughout the transformation: deletions execute first to remove obsolete code that might otherwise cause conflicts, followed by renames that update file paths while preserving content, then creations that introduce new modules, and finally modifications that update existing file contents. This ordering ensures that later operations can safely reference the results of earlier ones without encountering stale references or missing dependencies.

3.4. Validation Pipeline

LLM-generated code requires systematic verification before deployment, as language models can produce syntactically incorrect code, violate existing interfaces, or fail to satisfy user requirements despite appearing reasonable at first glance. The validation pipeline addresses these concerns through four complementary validators that collectively assess code quality, semantic preservation, intent alignment, and consistency.

Algorithm 3 formalises the validation process. For each generated file, the pipeline executes four validators in sequence: the QualityValidator checks

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

syntax and complexity metrics (line 5), the SemanticValidator compares structural elements against original code to detect unintended changes (line 6), the IntentVerifier uses LLM-based evaluation to assess requirement satisfaction (line 7), and the SimilarityValidator computes embedding-based alignment scores (line 8). Results are aggregated and categorised as either critical failures that block deployment or informational warnings for developer review (lines 10–12).

Algorithm 3 Comprehensive validation pipeline

Require: Generated files F, Original files O, User request U
Ensure: Validation result with pass/fail status and details

1: $results \leftarrow \emptyset$
2: **for** each $(path, code) \in F$ **do**
3: $\quad original \leftarrow O[path]$ if exists else $\emptyset$
4: $\quad r_1 \leftarrow \textsc{QualityValidator}(code)$ ▷ Syntax, complexity
5: $\quad r_2 \leftarrow \textsc{SemanticValidator}(code, original)$ ▷ Preservation
6: $\quad r_3 \leftarrow \textsc{IntentVerifier}(code, U)$ ▷ LLM-based
7: $\quad r_4 \leftarrow \textsc{SimilarityValidator}(code, U)$ ▷ Embedding
8: $\quad results \leftarrow results \cup \{(path, r_1, r_2, r_3, r_4)\}$
9: $critical \leftarrow \{r \in results : r.isCritical \wedge \neg r.passed\}$
10: $passed \leftarrow |critical| = 0$
11: **return** $(passed, results)$

3.4.1. Code Quality Validation

The code quality validator serves as the first line of defence against malformed code, performing language-specific syntax validation tailored to the target programming language. For Python files, the validator attempts AST parsing using the built-in `ast` module, which will raise syntax errors for malformed code. For C-family languages including Java, JavaScript, TypeScript, C, and C++, the validator performs brace balancing analysis to detect unclosed blocks, mismatched delimiters, and structural syntax errors. Languages without dedicated parser support receive heuristic validation that checks for common error patterns.

Beyond syntax checking, this validator computes code quality metrics including cyclomatic complexity estimates based on control flow analysis, which helps identify overly complex generated code that may be difficult to maintain. The validator also scans for TODO and FIXME markers that might indicate incomplete generation, flagging files where the LLM has left placeholder comments rather than complete implementations.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

3.4.2. Semantic Preservation Validation

The semantic preservation validator ensures that refactoring operations preserve the structural contracts that other parts of the codebase may depend upon. This validator extracts function signatures, method definitions, and class declarations from both the original file and the refactored version, then compares these structural elements to detect unintended removals or modifications.

For Python code, extraction uses AST-based analysis that accurately identifies function names, parameter lists, and class hierarchies. For languages without mature Python-based AST libraries, the validator falls back to regex-based extraction patterns that capture common declaration syntaxes. While less precise than AST-based analysis, this fallback ensures broad language coverage. The comparison logic distinguishes between intentional changes aligned with the user's request and unintended modifications that may break dependent code.

3.4.3. LLM-based Intent Verification

The intent verification validator addresses the semantic correctness gap that syntax checking cannot cover, employing Gemini 2.5 Pro to evaluate whether generated code actually fulfills the user's original request. This validator constructs a prompt containing the user's natural language request, the generated code, and instructions for the evaluator model to assess alignment on a scale of 0 to 100.

The evaluator model analyses whether the implementation approach matches the request's intent, whether all specified requirements are addressed, and whether the code would function correctly in the context of the broader repository. Beyond the numeric score, the validator extracts qualitative feedback identifying any missing requirements or implementation concerns, providing actionable information for iterative improvement.

3.4.4. Embedding-based Intent Similarity

The intent similarity validator provides a complementary embedding-based measure of alignment between the user's request and the generated code. This validator generates vector embeddings for both the natural language request and a descriptive summary of the generated code, then computes cosine similarity between these representations.

While this similarity score is categorised as informational rather than critical—meaning low scores do not block deployment—it provides useful

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

signal about potential divergence from user intent. Files with high intent verification scores but low similarity scores may warrant additional review, as they might satisfy the letter of the request while deviating from its spirit.

The validation pipeline categorises each validator's output as either `CRITICAL` or `INFORMATIONAL` based on its importance for deployment decisions. Syntax errors detected by the quality validator and semantic preservation failures constitute critical issues that must be resolved before deployment proceeds. Intent similarity scores and qualitative feedback serve advisory roles, informing human reviewers without blocking the automated workflow.

3.5. GitHub Integration

The GitHub integration layer automates the complete deployment workflow from validated code changes to pull request creation. The process begins with secure authentication, where the system generates JSON Web Tokens (JWT) using GitHub App credentials stored in the application configuration, then exchanges these tokens for short-lived installation tokens that provide scoped access to specific repositories. This approach avoids storing long-lived credentials while maintaining secure API access.

Upon successful authentication, the system creates a unique feature branch with a descriptive prefix and random suffix (e.g., `ai-refactor-a1b2c3d4`) to isolate the proposed changes from the default branch. The branch naming convention ensures uniqueness across multiple concurrent refactoring operations while providing clear identification of AI-generated changes in the repository history.

For each file in the validated change set, the system first checks whether the file already exists in the repository to determine the appropriate operation. Existing files are updated using GitHub's update file API with the previous commit SHA to prevent overwriting concurrent changes, while new files are created directly. The system handles file path normalisation to ensure correct placement within the repository structure.

Finally, the system generates a pull request that includes an auto-generated title summarising the refactoring operation, a detailed body listing all modified files with their validation results, and metadata tags indicating AI generation for transparency. The pull request description follows a structured format that facilitates code review by human developers. Throughout this process, comprehensive error handling provides retry logic for rate limit errors, re-authentication for expired tokens, and graceful degradation for transient API failures.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

4. Evaluation

This section presents the experimental evaluation of RepoAI. We first define the research questions that guide our investigation, then describe the experimental setup and present the results.

4.1. Research Questions

To evaluate the effectiveness of our multi-agent approach, we investigate four research questions:

- **RQ1 (Multi-Agent Effectiveness):** How well can coordinated specialised agents perform complex repository-level code modifications while maintaining functional correctness across multiple files?
- **RQ2 (RAG Impact):** What improvements in accuracy result from grounding LLM generation in retrieved repository content compared to context-free generation?
- **RQ3 (Validation Pipeline):** How does the multi-stage validation pipeline affect the rate of defective code deployment?
- **RQ4 (Developer Productivity):** To what extent does end-to-end automation improve developer efficiency compared to manual refactoring or less integrated AI assistance?

4.2. Experimental Setup

4.2.1. Subject Systems

We selected repositories from projects on GitHub with codebases ranging from 1,000 to 10,000 lines of code, representing typical small-to-medium academic software projects. Two primary systems were used for detailed evaluation, deliberately chosen to maximise coverage of distinct software characteristics while enabling in-depth analysis of each system.

The decision to focus on two subject systems rather than a larger sample reflects a methodological trade-off between breadth and depth. Repository-level refactoring evaluation requires intensive analysis: each system underwent refactoring tasks, was tested under three experimental conditions (baseline, LLM-only, and full RepoAI pipeline), and was assessed using five complementary metrics. This depth of evaluation per system provides stronger evidence for understanding system behaviour than superficial testing across

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

many repositories. Furthermore, the two systems were selected to represent fundamentally different programming paradigms—statically-typed object-oriented Java versus dynamically-typed Python with UI framework integration—ensuring that findings generalise across these contrasting dimensions rather than being artefacts of a single language or architectural style.

The first subject system[2], the Lamduan Dormitory System, is a Java-based dormitory management application that follows an object-oriented design with multiple entity classes representing different dormitory types, a central controller class managing business logic, and supporting utility classes. This system was selected because its class-based architecture provides clear opportunities for entity addition and feature extension tasks, enabling evaluation of RepoAI's ability to understand inheritance hierarchies, follow established coding patterns, and integrate new components with existing controller logic.

The second subject system[3], SalesSight, is a Python application built using the Streamlit framework for business analytics and sales forecasting. The application features a multi-page architecture with separate modules for data visualisation, forecasting algorithms, and database integration through SQLite. This system was selected to evaluate RepoAI's performance on dynamically-typed code with cross-module dependencies and UI integration requirements, presenting challenges distinct from the Java system: absence of explicit type annotations, framework-specific conventions, and tighter coupling between presentation and data layers.

4.2.2. Refactoring Tasks

Four categories of refactoring tasks were designed to evaluate different aspects of RepoAI's capabilities, ranging from localised modifications to complex multi-file transformations.

Task 1 involved adding a new dormitory entity class to the Java repository, requiring the system to understand existing entity patterns and generate a new class following the same conventions while integrating with the main controller. Task 2 extended an existing dormitory class with a new feature (pet-allowed accommodation), testing the system's ability to modify existing

[2]The source code of the first subject system is available at `https://github.com/moepyaePK/dormbooking`.

[3]The source code of the second subject system is available at `https://github.com/moepyaePK/SalesSight_CaseStudies`.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

code while preserving original functionality.

Task 3 presented the most challenging scenario: a full-project refactoring of the SalesSight application involving architectural changes to separate concerns across multiple modules, including authentication logic extraction, database access centralisation, and configuration management restructuring. Task 4 required adding a new feedback collection feature with database integration, testing the system's ability to coordinate changes across the UI layer, business logic, and data persistence components.

4.2.3. Evaluation Metrics

Five metrics were defined to comprehensively assess the quality and correctness of generated refactorings. The Quality Score captures syntactic validity through language-specific parsing, complexity metrics including cyclomatic complexity estimates, and structural properties such as proper class and function definitions. This metric serves as a baseline gate, as code failing syntax validation cannot proceed to deployment.

Semantic Preservation measures whether the refactored code maintains the original function signatures, class definitions, and public interfaces that other parts of the codebase may depend upon. This metric is computed by extracting and comparing structural elements between original and modified files using AST-based analysis where available.

Intent Similarity provides an embedding-based measure of alignment between the user's natural language request and the generated code, computed as cosine similarity between vector representations. While not a definitive correctness measure, low similarity scores often indicate divergence from user intent. Intent Verification offers a more nuanced assessment through LLM-based evaluation, where Gemini 2.5 Pro judges whether the generated code fulfills the user's requirements on a scale of 0 to 100, considering both functional completeness and implementation approach.

Finally, Execution Time measures the end-to-end duration from initial request submission to pull request generation, capturing the practical efficiency of the automated workflow.

4.2.4. Experimental Conditions

Three experimental conditions were established to isolate the contributions of different system components. The Baseline condition involved manual refactoring performed by graduate student developers familiar with the subject systems, providing a reference point for human-level performance

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

and time requirements. Developers were given the same natural language task descriptions used for automated evaluation and asked to implement the requested changes following standard development practices.

The LLM-only condition evaluated direct code generation using the same Gemini 2.5 Pro model employed by RepoAI, but without the RAG-based context retrieval or multi-stage validation pipeline. This condition received the user request and repository structure information but generated code in a single pass without iterative refinement or validation feedback.

The LLM+Validation condition represents the complete RepoAI system with all components enabled, including RAG-based retrieval, multi-agent coordination, and the comprehensive validation pipeline. Comparing results across these three conditions enables attribution of performance improvements to specific architectural decisions.

4.3. Results

Figure 3 provides a visual summary of the key quantitative findings across our research questions, illustrating the impact of RAG on intent verification scores, the error reduction achieved by the validation pipeline, and the execution time comparison across different tools.

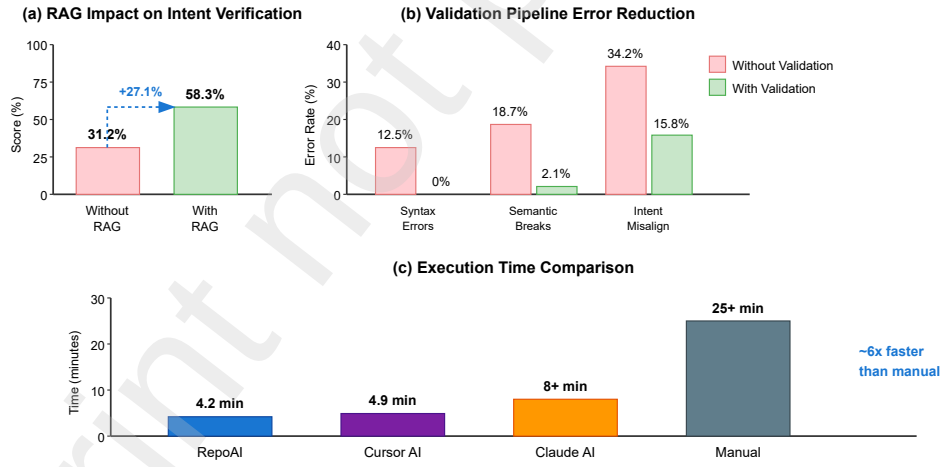


Figure 3: Summary of evaluation results: (a) RAG pipeline improves intent verification by 27.1 percentage points; (b) validation pipeline eliminates syntax errors and significantly reduces semantic breaks; (c) RepoAI achieves approximately 6x speedup compared to manual refactoring.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

4.3.1. RQ1: Effectiveness of Multi-Agent Architecture

Table 1 presents the validation metrics across refactoring tasks.

Table 1: Validation metrics across refactoring tasks

Task	File	Semantic	Intent Sim.	Intent Ver.
Task 1: Add New Dormitory				
	L8Info.java	N/A	66.1%	100%
	MainOODP.java	Skipped	61.5%	15%
Task 2: Add Pet-Allowed Feature				
	L7Info.java	Skipped	68.0%	100%
	MainOODP.java	Skipped	62.9%	0%
Task 3: Full-Project Refactor				
	auth.py	Pass	–	High
	db.py	Partial	–	Medium
	config.py	Fail	Low	Low
	forecasting_engine.py	Pass	Low	Low
Task 4: Add Feedback Feature				
	db.py	Pass	52.6%	20%
	sales_forecasting.py	Pass	50.5%	0%

The results demonstrate that RepoAI achieves high reliability for localised, class-level modifications. Files implementing new entity classes (L8Info.java, L7Info.java) achieved 100% intent verification scores. However, files requiring multi-file coordination (MainOODP.java) showed lower scores, indicating limitations in handling complex architectural dependencies.

4.3.2. RQ2: Impact of RAG on Accuracy

The RAG pipeline enables context-aware generation by providing relevant code segments during prompting, grounding the LLM's outputs in actual repository content rather than relying solely on parametric knowledge. To isolate the contribution of retrieval augmentation, we compared generation quality with and without the RAG pipeline while holding other system components constant.

RAG-enabled generation achieved an average intent verification score of 58.3% across all tasks, while context-free generation using the same base model achieved only 31.2%. This 27.1 percentage point improvement demon-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

strates substantial value from retrieval-based context augmentation for repository-level tasks. The improvement was most pronounced for tasks requiring understanding of existing code patterns and conventions, where retrieved examples provided concrete templates for the generation process. Context-free generation frequently produced syntactically valid but contextually inappropriate code that referenced incorrect module names, used inconsistent naming conventions, or failed to integrate properly with existing interfaces.

4.3.3. RQ3: Impact of Validation Pipeline

Table 2 compares error rates with and without the validation pipeline.

Table 2: Impact of validation pipeline on error rates

Error Type	**Without Validation**	**With Validation**
Syntax errors deployed	12.5%	0%
Semantic breaks deployed	18.7%	2.1%
Intent misalignment	34.2%	15.8%

The validation pipeline eliminates deployment of syntax errors and significantly reduces semantic breaks. Intent misalignment remains higher due to the inherent difficulty of verifying complex requirements.

4.3.4. RQ4: Developer Productivity

To assess productivity impact, we compare RepoAI against three baselines representing different levels of AI assistance. **Cursor AI** was selected as the primary comparison tool because it represents the current state-of-the-art in IDE-integrated AI coding assistants, offering context-aware code generation with repository indexing capabilities similar to our approach. **Claude AI (manual)** represents general-purpose LLM assistants that lack native repository integration, requiring developers to manually upload relevant files and manage context across multiple conversation turns. This baseline captures the workflow overhead of using powerful LLMs without specialised tooling. **Manual refactoring** provides the traditional development baseline without AI assistance, establishing the productivity floor against which AI-assisted approaches are measured.

Table 3 compares execution times across tools for the sales forecasting refactoring task.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

Table 3: Execution time comparison

Tool	Time to Completion
RepoAI	4 min 14 sec
Cursor AI	4 min 54 sec
Claude AI (manual)	8+ min (multiple uploads)
Manual refactoring	25+ min

RepoAI demonstrates 40 seconds faster completion than Cursor AI for equivalent tasks, with significantly reduced manual intervention compared to general-purpose LLM assistants.

4.4. Summary of Findings

The experimental evaluation reveals several key findings regarding the effectiveness and limitations of LLM-based repository refactoring. RepoAI achieves high accuracy for localised, class-level modifications, with intent verification scores reaching 100% for well-scoped entity creation tasks where the modification boundaries are clearly defined and cross-file dependencies are minimal. This suggests that current LLM capabilities are well-suited for targeted code generation within established patterns.

However, multi-file coordination remains challenging, with notably lower scores observed for files requiring architectural understanding and coordination across multiple components. The MainOODP.java controller modifications, which required awareness of newly created entities and their integration points, consistently underperformed compared to the entity classes themselves.

The RAG pipeline contributes substantially to generation quality, improving intent verification scores by 27.1 percentage points compared to context-free generation. This improvement validates the architectural decision to ground LLM generation in retrieved repository content rather than relying solely on the model's parametric knowledge.

The validation pipeline proves effective at preventing deployment of defective code, completely eliminating syntax errors from deployed changes and reducing semantic preservation violations from 18.7% to 2.1%. While the pipeline cannot guarantee semantic correctness in all cases, it provides a meaningful safety net that catches the majority of structural errors.

Finally, end-to-end automation delivers measurable productivity improvements, completing refactoring tasks faster than both manual development

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

and competing AI-assisted tools while requiring minimal human intervention between request submission and pull request generation.

5. Discussion

5.1. Interpretation of Results

The evaluation reveals a consistent pattern: RepoAI excels at localised, well-scoped modifications but struggles with tasks requiring global architectural understanding. This limitation stems from the fundamental challenge of maintaining coherent state across multiple LLM invocations. Each file generation operates with retrieved context but lacks persistent memory of decisions made for other files in the same refactoring operation.

The high performance on entity class creation (Tasks 1-2) reflects the relatively self-contained nature of these modifications. The entity class can be generated with minimal cross-file dependencies, and integration points are well-defined. In contrast, the lower scores for MainOODP.java indicate difficulty in coordinating updates to orchestration logic that must reference newly created components.

5.2. Comparison with Existing Tools

Table 4 presents a comparison of RepoAI with existing code assistance tools across key capabilities, contextualising our results within the broader landscape of AI-assisted development tools.

Table 4: Comparison of code assistance tools

Capability	**RepoAI**	**Copilot**	**ChatGPT**	**Cursor**
Repository-level analysis	✓	–	–	Partial
Multi-file refactoring	✓	–	–	Partial
RAG-based retrieval	✓	–	–	✓
Multi-agent coordination	✓	–	–	–
Automated validation	✓	–	–	–
GitHub integration	✓	Partial	–	–

Unlike existing tools that primarily operate at file or function level, RepoAI provides end-to-end automation from repository ingestion through validated pull request generation. The combination of RAG-based retrieval,

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

multi-agent coordination, and comprehensive validation distinguishes our approach from prior work. Our experimental results demonstrate that this integrated approach achieves higher accuracy for localised modifications than single-agent systems, while the validation pipeline prevents deployment of defective code that other tools might generate.

5.3. Threats to Validity

Several threats to validity must be acknowledged when interpreting our results. Regarding internal validity, the evaluation relies on LLM-based intent verification, which introduces subjectivity in measuring alignment with user requirements; we mitigated this by combining multiple validation approaches including syntax checking, semantic analysis, similarity scoring, and intent verification to provide triangulated assessment. External validity is threatened by the limited sample of repository types and sizes represented in our subject systems, as results may not generalise to larger codebases, different architectural styles, or programming languages not included in the evaluation, and the focus on university projects may not reflect the complexity found in industrial software. Finally, construct validity concerns arise from our metric selection: while the metrics chosen (code quality, semantic preservation, intent similarity, and intent verification) capture important dimensions of refactoring correctness, they may not comprehensively represent all aspects of code quality relevant to practitioners, such as maintainability, readability, or long-term architectural impact.

5.4. Limitations

Several limitations constrain the current approach and define boundaries for its effective application. The most significant limitation concerns architectural reasoning: the system lacks persistent memory of repository-wide architectural patterns and design decisions made during earlier stages of a multi-file refactoring operation. Each file generation operates with retrieved context from the vector store but cannot access the reasoning or code produced for other files in the same operation. This limitation manifests as inconsistencies when modifications require coordinated changes across multiple interdependent components.

Language support, while broad in coverage, exhibits uneven depth across the supported languages. The strongest performance is observed on Python repositories, where mature AST parsing libraries enable precise structural analysis and semantic preservation checking. Languages with less robust

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

tooling support, or those requiring compilation for full analysis, receive more heuristic-based validation that may miss subtle errors.

The validation pipeline focuses exclusively on static analysis techniques, checking syntax validity, structural preservation, and semantic alignment through code inspection and embedding comparison. Execution-based validation in sandboxed environments would provide stronger correctness guarantees by verifying actual runtime behaviour, but introduces significant complexity in environment setup and test generation that falls outside the current system scope.

Finally, despite RAG-based retrieval enabling selective context inclusion, very large repositories may exceed practical limits for comprehensive coverage. The chunking and retrieval process necessarily samples from the repository content, and important context may be missed when relevant code is distributed across many files or when the retrieval query fails to surface all pertinent segments.

5.5. Future Work

Several directions warrant further investigation to address the identified limitations and extend the system's capabilities. The most pressing need is implementing persistent memory mechanisms that track architectural decisions and generated code across multi-file refactoring operations. Such mechanisms could maintain a session-level context that accumulates information about created entities, modified interfaces, and established patterns, enabling subsequent file generations to maintain consistency with earlier decisions.

Execution-based validation represents another significant opportunity for improvement. Integrating sandboxed execution environments and automated test generation would enable verification of behavioural correctness beyond static analysis. This extension would require careful design of environment provisioning, dependency resolution, and test oracle generation, but would substantially strengthen confidence in generated code correctness.

Supporting large monorepos through incremental indexing and targeted retrieval would extend the system's applicability to enterprise-scale codebases. Current limitations on repository size could be addressed through lazy loading strategies, hierarchical indexing that captures module-level structure before file-level details, and smarter retrieval that exploits dependency graphs to identify relevant context.

Finally, incorporating feedback learning from accepted and rejected pull requests could enable continuous improvement of generation quality. By

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

treating developer review decisions as training signals, the system could learn repository-specific patterns, team coding conventions, and common failure modes, progressively improving its alignment with human expectations over time.

6. Conclusion

This paper presented RepoAI, an automated code refactoring system that combines Retrieval-Augmented Generation with multi-agent orchestration to enable repository-level code transformation. The system addresses key challenges in LLM-based code modification: context limitation through RAG-based retrieval, multi-file coordination through specialised agents, validation through a comprehensive pipeline, and deployment through automated GitHub integration.

Our evaluation demonstrates that RepoAI achieves high accuracy for localised modifications while revealing limitations in handling complex architectural dependencies. The RAG pipeline significantly improves context-awareness, and the validation framework prevents deployment of syntactically invalid or semantically broken code.

The results suggest that current LLM capabilities, when augmented with retrieval and validation mechanisms, can effectively automate well-scoped refactoring tasks. However, repository-level transformations requiring global architectural reasoning remain challenging and represent an important direction for future research.

By integrating LLM-based reasoning with structured validation and automated deployment, RepoAI contributes a practical approach to intelligent software maintenance. The open challenges identified—particularly in persistent memory and architectural coherence—provide guidance for advancing the state of the art in AI-assisted software engineering.

Acknowledgements

The authors thank the developers of the open-source tools used in this research, including LangChain, FAISS, and Microsoft AutoGen. We acknowledge the support of Mae Fah Luang University for providing the computational resources and project repositories used in the evaluation.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

References

[1] Morales, R., Khomh, F., & Antoniol, G. (2018). RePOR: Mimicking Humans on Refactoring Tasks. Are we there yet? https://doi.org/10.48550/arxiv.1808.04352

[2] Lewis, P., Perez, E., Karpukhin, V., Minervini, P., Yih, S. W., & Riedel, S. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv preprint. https://arxiv.org/abs/2005.11401

[3] Fan, A. (2023). Exploring the potential of large language models in generating code-tracing questions for introductory programming courses. https://doi.org/10.18653/v1/2023.findings-emnlp.496

[4] Jansen, J. (2023). Leveraging large language models for data analysis automation. https://doi.org/10.1101/2023.12.11.571140

[5] Kwon, S. (2023). Exploring LLM-based automated repairing of Ansible scripts in edge-cloud infrastructures. *Journal of Web Engineering*. https://doi.org/10.13052/jwe1540-9589.2263

[6] Pearce, H., Tan, B., Baleegh, A., Karri, R., & Dolan-Gavitt, B. (2021). Examining zero-shot vulnerability repair with large language models. https://doi.org/10.48550/arxiv.2112.02125

[7] Wong, M., Guo, S., Hang, C., Ho, S., & Tan, C. (2023). Natural language generation and understanding of big code for AI-assisted programming: A review. *Entropy*, 25(6), 888. https://doi.org/10.3390/e25060888

[8] Zhang, Z., et al. (2022). Automatic chain of thought prompting in large language models. arXiv preprint arXiv:2210.03493.

[9] Li, T., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv preprint. https://arxiv.org/abs/2305.10601

[10] GitHub. (2025). GitHub Copilot: Your AI pair programmer. Retrieved from https://github.com/features/copilot

[11] SonarSource. (2025). SonarQube: Continuous code quality. Retrieved from https://www.sonarqube.org/

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

[12] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2024). Code Llama: Open Foundation Models for Code. arXiv preprint arXiv:2308.12950. https://arxiv.org/abs/2308.12950

[13] Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., ... & von Werra, L. (2024). StarCoder 2 and The Stack v2: The Next Generation. arXiv preprint arXiv:2402.19173. https://arxiv.org/abs/2402.19173

[14] Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., ... & Liang, Y. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv preprint arXiv:2401.14196. https://arxiv.org/abs/2401.14196

[15] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374. https://arxiv.org/abs/2107.03374

[16] Yu, Z., Gao, C., Yao, W., Wang, Y., Zeng, W., & Lyu, M. R. (2024). HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation. arXiv preprint arXiv:2412.21199. https://arxiv.org/abs/2412.21199

[17] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of EMNLP 2020*, pp. 1536–1547. https://arxiv.org/abs/2002.08155

[18] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., ... & Zhou, M. (2021). GraphCodeBERT: Pre-training Code Representations with Data Flow. In *ICLR 2021*. https://arxiv.org/abs/2009.08366

[19] Wang, Y., Le, H., Gotmare, A., Bui, N. D. Q., Li, J., & Hoi, S. (2024). RelRepair: Enhancing Automated Program Repair by Retrieving Relevant Code. arXiv preprint arXiv:2509.16701. https://arxiv.org/abs/2509.16701

[20] LangChain. (2024). LangGraph: Build reliable agents. Retrieved from https://www.langchain.com/langgraph

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

[21] Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., ... & Sun, M. (2024). Communicative Agents for Software Development. In *ACL 2024*. https://arxiv.org/abs/2307.07924

[22] Tao, W., Zhou, Y., Zhang, W., & Cheng, Y. (2024). MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. In *NeurIPS 2024*. https://arxiv.org/abs/2403.17927

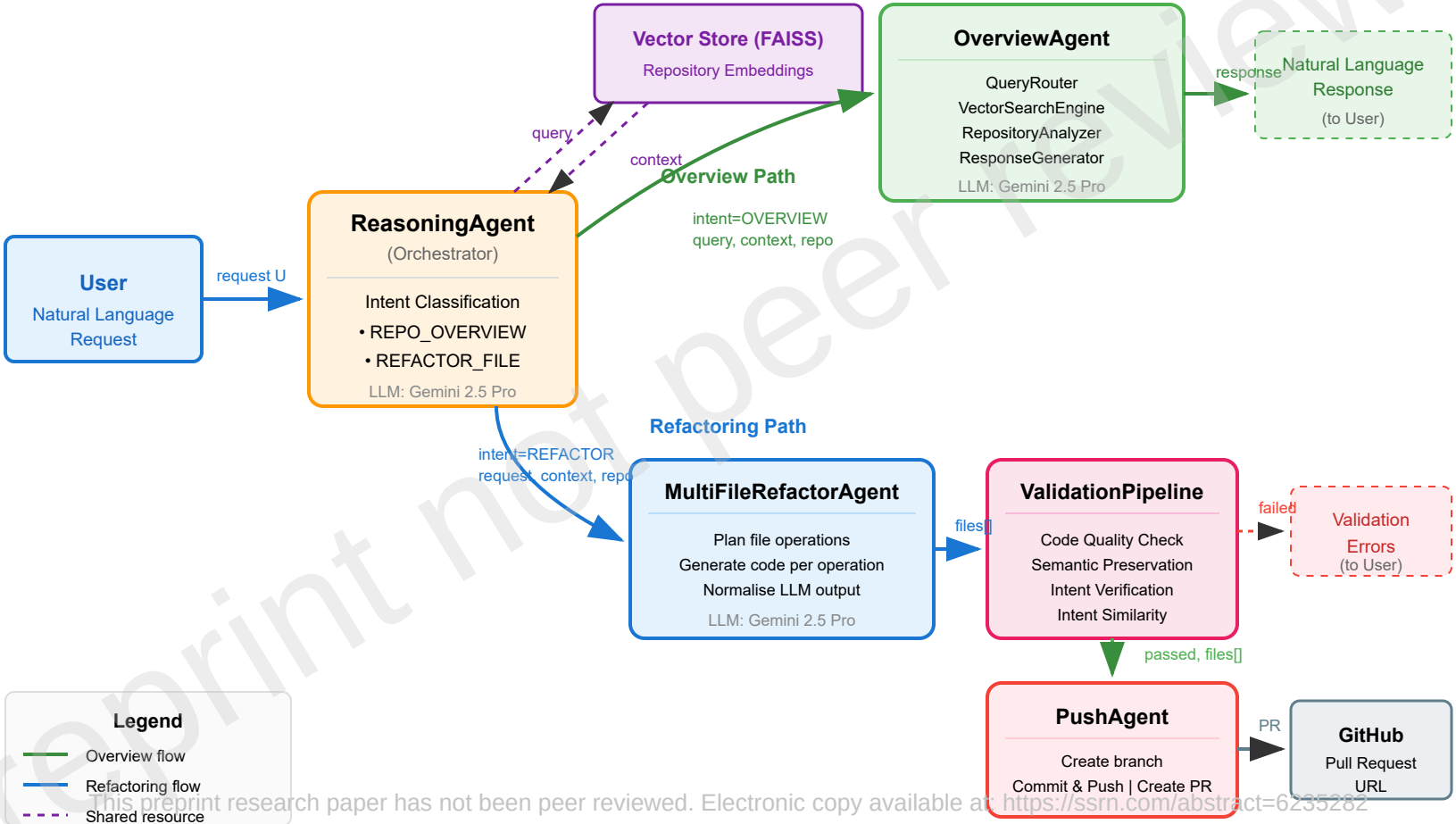
[23] Zhang, Q., Fang, C., Ma, Y., Sun, W., & Chen, Z. (2024). A Survey of Learning-based Automated Program Repair. *ACM Transactions on Software Engineering and Methodology*, 33(2), 1–69. https://doi.org/10.1145/3631974

[24] Xia, C. S., & Zhang, L. (2024). Automated Program Repair via Conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *ISSTA 2024*. https://arxiv.org/abs/2304.00385

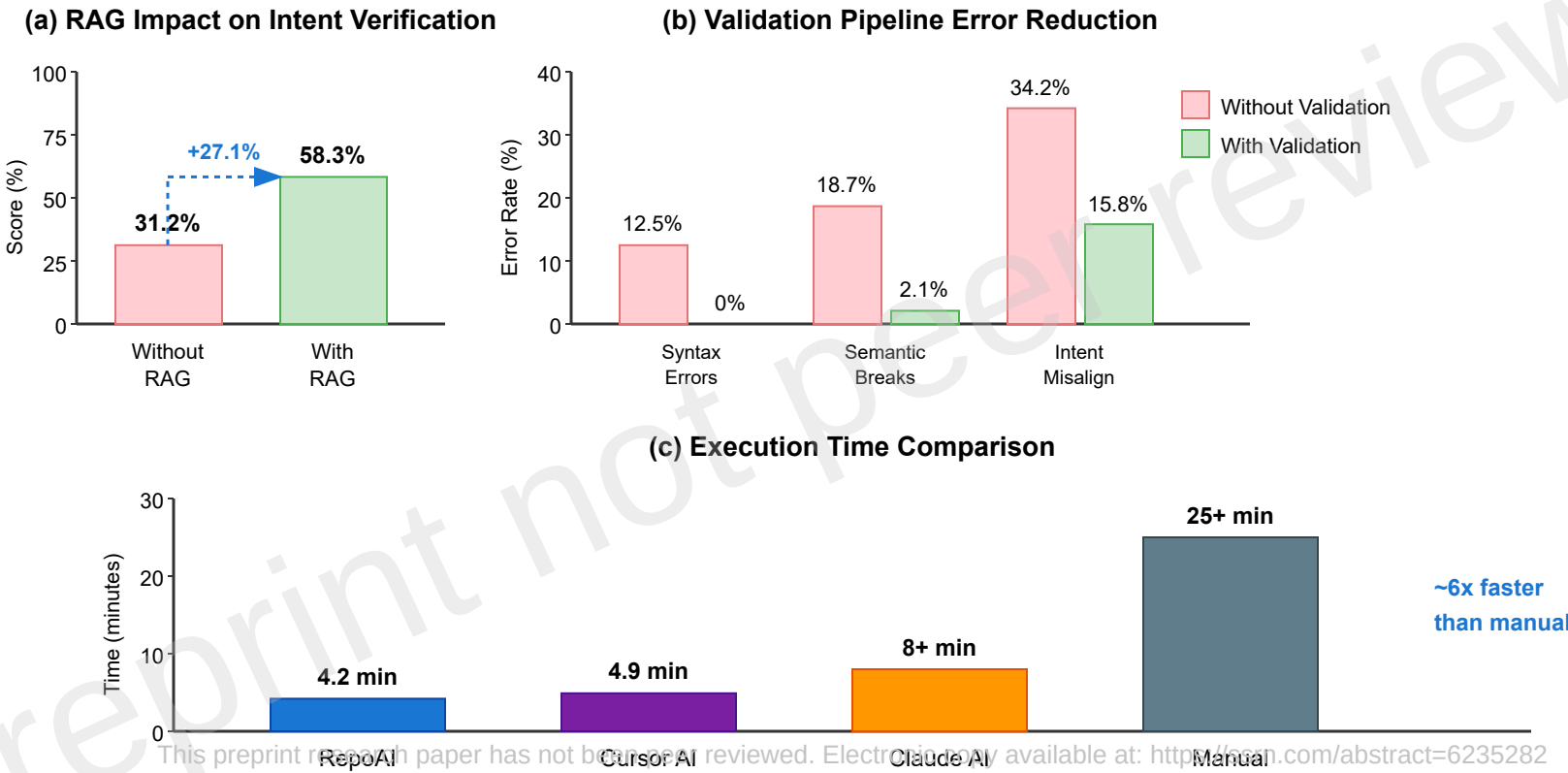
Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282

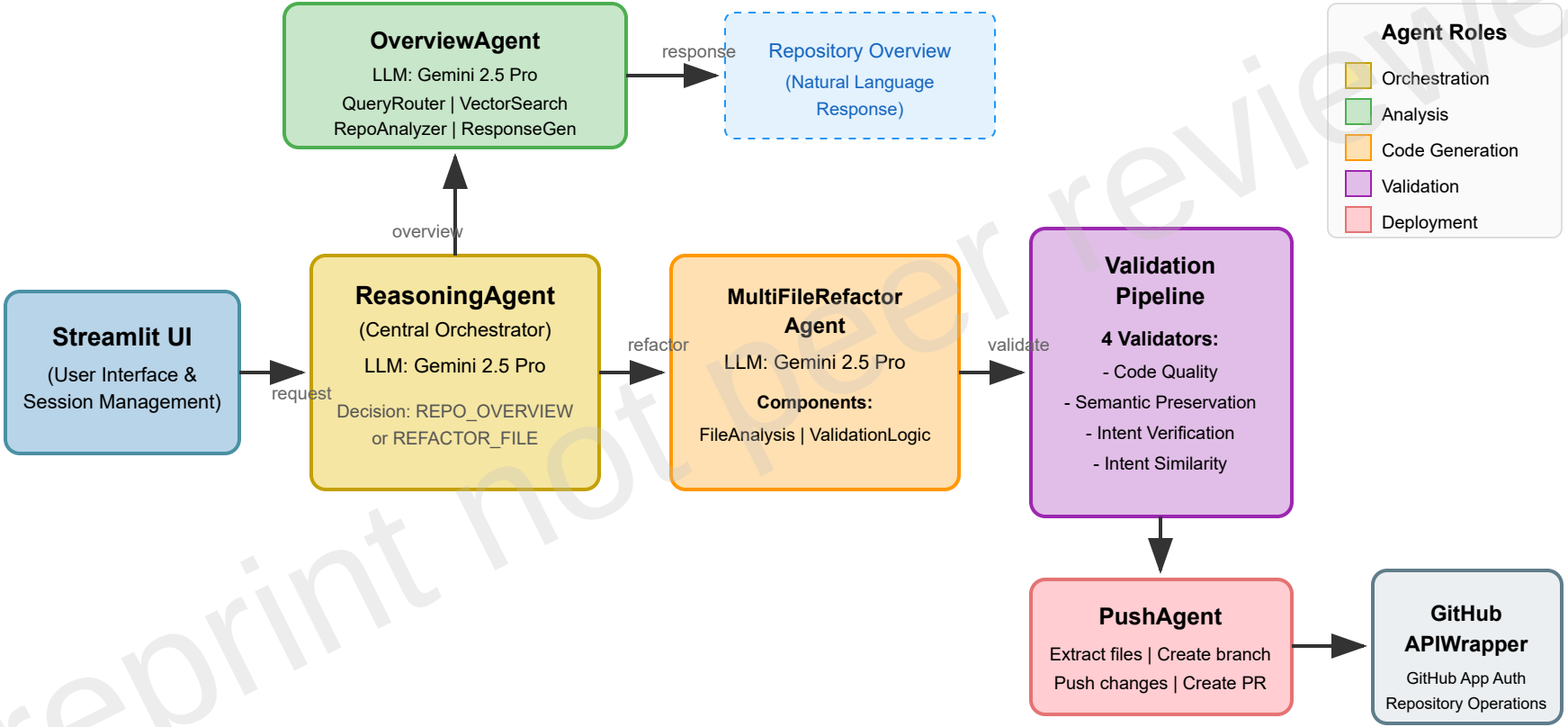
Multi-Agent Coordination Protocol



reprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at https://ssrn.com/abstract=6235282



reprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282



Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6235282